



Web Services SOAP et RESTful

Mohamed Youssfi : med@youssfi.net
ENSET, Université Hassan II Mohammeda

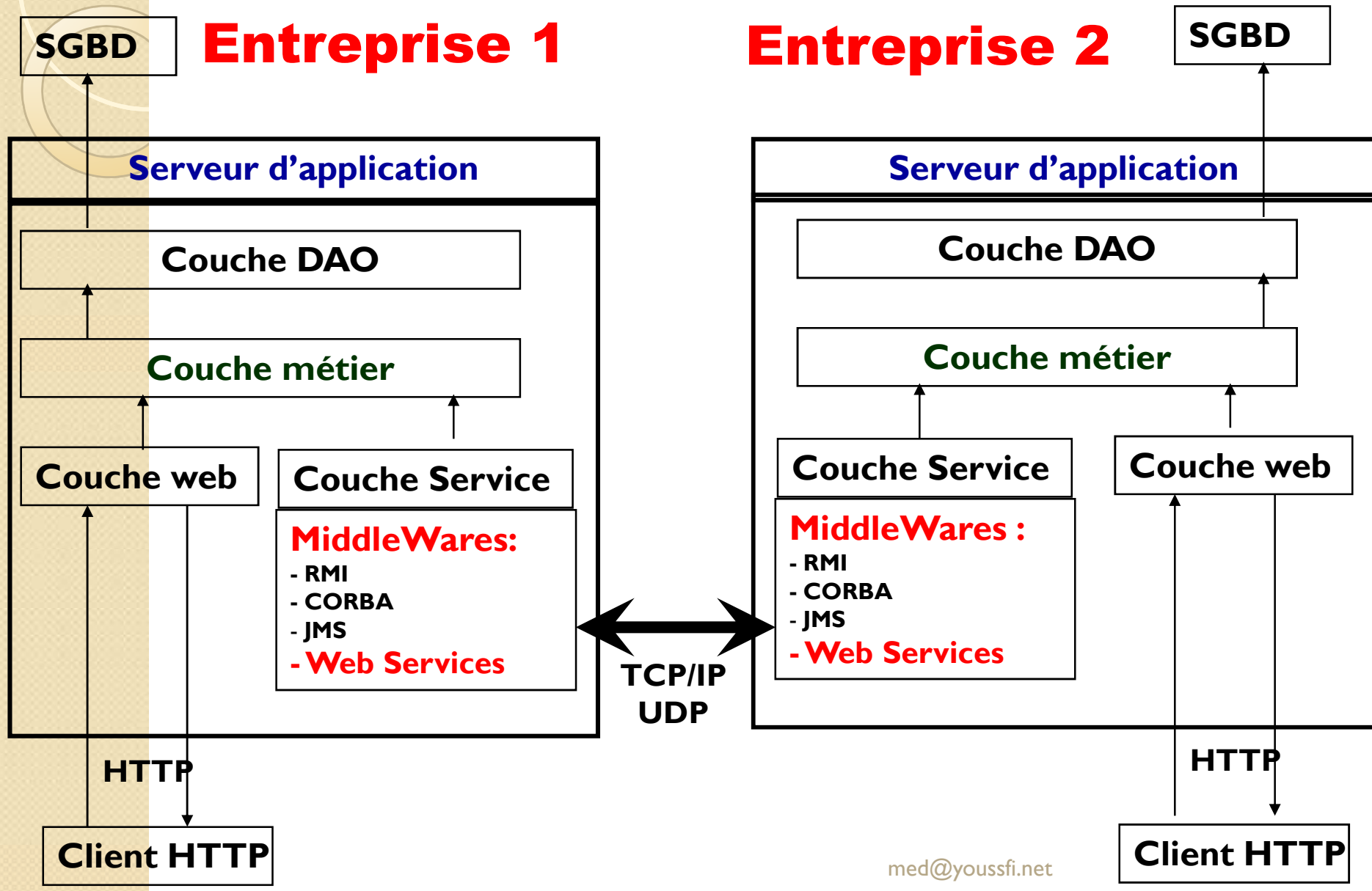
Exigences d'un projet informatique

- Exigences fonctionnelles:
 - Une application est créée pour répondre , tout d'abord, aux besoins fonctionnels des entreprises.
- Exigences Techniques :
 - Les performances:
 - Temps de réponse
 - Haute disponibilité et tolérance aux pannes
 - Eviter le problème de montée en charge
 - La maintenance:
 - Une application doit évoluer dans le temps.
 - Doit être fermée à la modification et ouverte à l'extension
 - Sécurité
 - Portabilité
 - **Distribution**
 - **Capacité de communiquer avec d'autres applications distantes.**
 - Capacité de fournir le service à différents type de clients (Desk TOP, Mobile, SMS, http...)
 -
 - Coût du logiciel

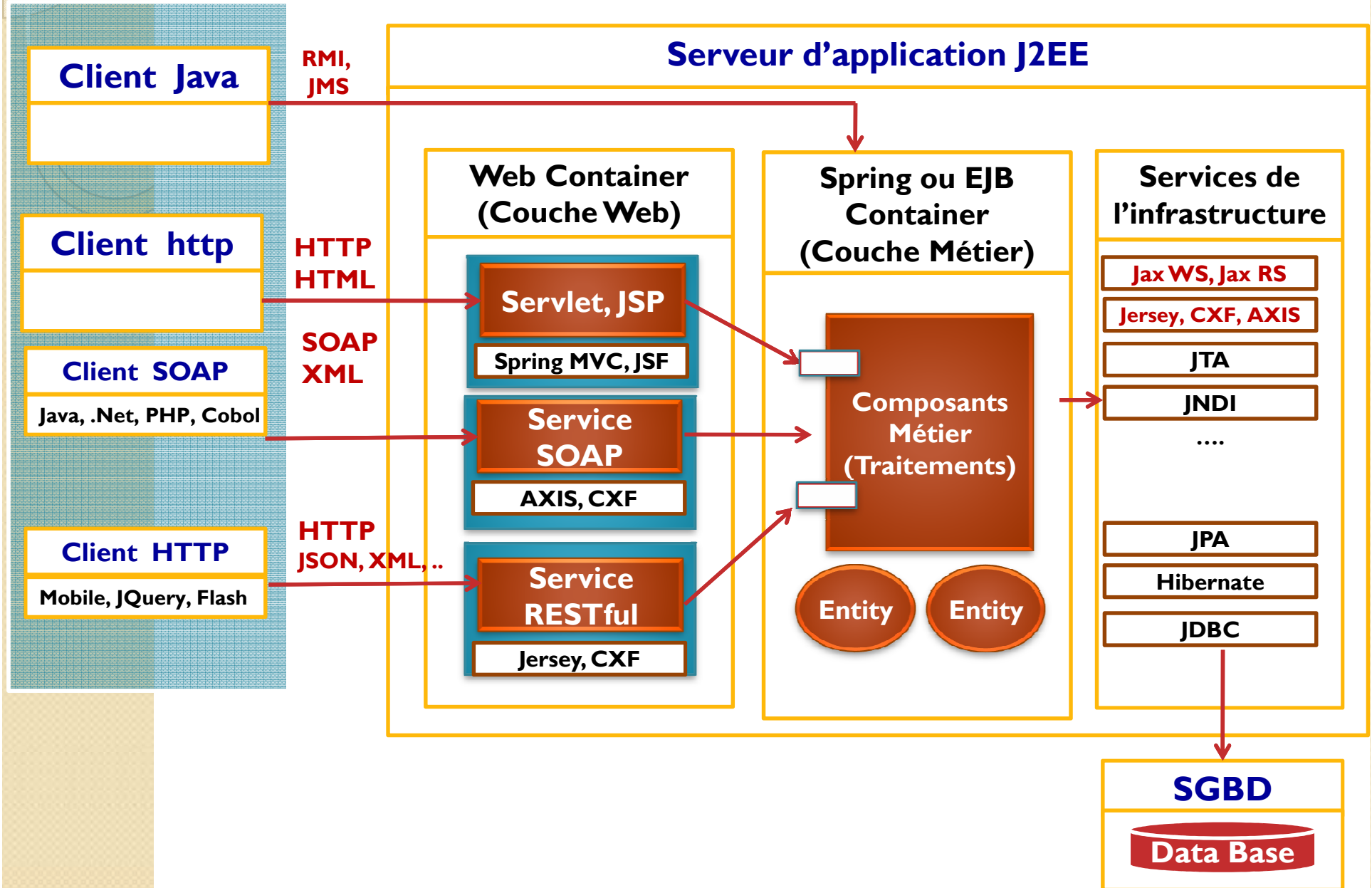
Constat

- Il est très difficile de développer un système logiciel qui respecte ces exigences sans **utiliser l'expérience des autres** :
 - Serveur d'application JEE:
 - JBOSS, Web Sphere,
 - GlassFish, Tomcat,
 - ...
 - Framework pour l'Inversion de contrôle:
 - Spring (Conteneur léger)
 - EJB (Conteneur lourd)
 - Frameworks :
 - Mapping objet relationnel (ORM) : JPA, Hibernate, Toplink, ...
 - Applications Web : Struts, JSF, SpringMVC
 -
 - Middlewares :
 - RMI, CORBA : Applications distribuées
 - **JAXWS pour Web services SOAP**
 - **JAXRS pour les Web services RESTful**
 - JMS : Communication asynchrone entre les application
 - ...

Vision globale d'une architectures Distribuées



Architecture J2EE





Web services

SOAP
WSDL

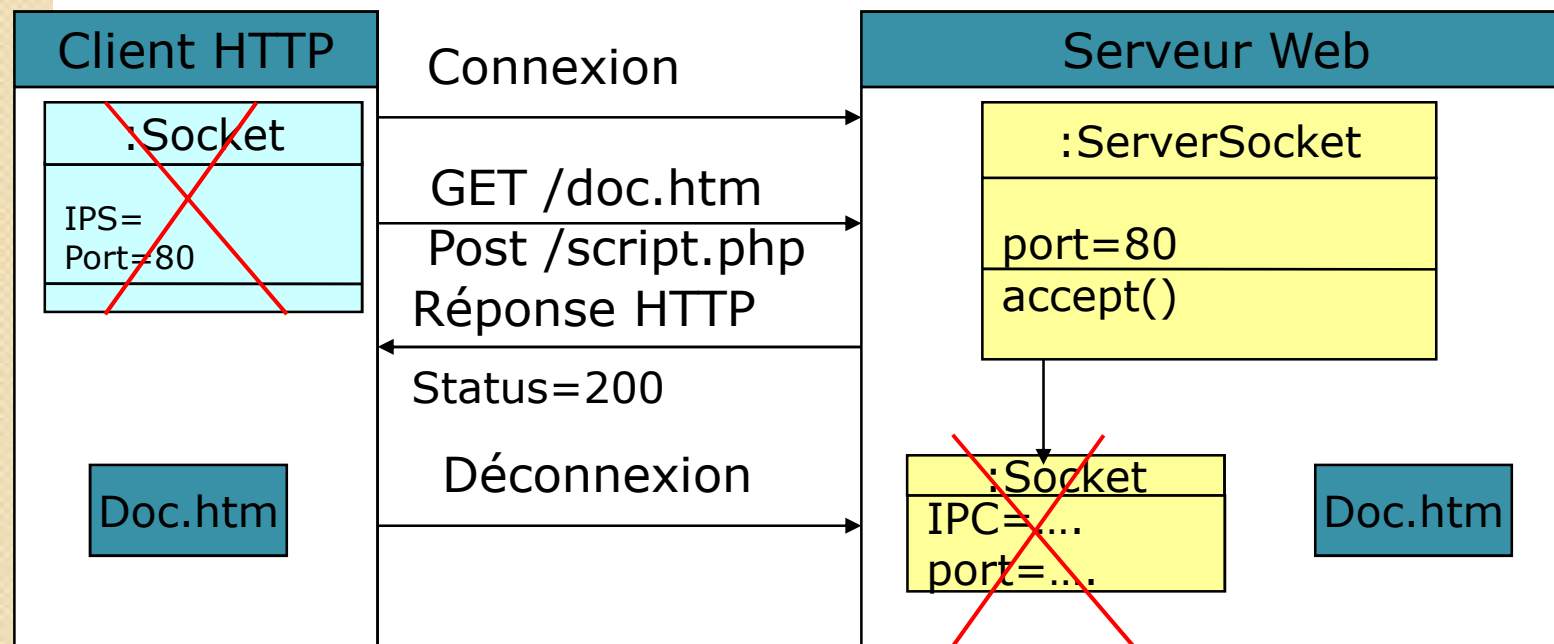
Introduction aux web services

- Les Web Services sont des composants web basés sur Internet (**HTTP**) qui exécutent des tâches précises et qui respectent un format spécifique (**XML**).
- Ils permettent aux applications de faire appel à des fonctionnalités à distance en simplifiant ainsi l'échange de données.
- Les Web Services permettent aux applications de dialoguer à travers le réseau, indépendamment de
 - leur plate-forme d'exécution
 - et de leur langage d'implémentation.
- Ils s'inscrivent dans la continuité d'initiatives telles que
 - CORBA (*Common Object Request Broker Architecture*, de l'OMG) en apportant toutefois une réponse plus simple, s'appuyant sur des technologies et standards reconnus et maintenant acceptés de tous.

LE PROTOCOLE HTTP

- **HTTP :HyperText Tranfert Protocol**
 - Protocole qui permet au client de récupérer des documents du serveur
 - Ces documents peuvent être statiques (contenu qui ne change pas : HTML, PDF, Image, etc..) ou dynamiques (Contenu généré dynamiquement au moment de la requête : PHP, JSP, ASP...)
 - Ce protocole permet également de soumissionner les formulaires
- **Fonctionnement (très simple en HTTP/1.0)**
 - Le client se connecte au serveur (Créer une socket)
 - Le client demande au serveur un document : Requête HTTP
 - Le serveur renvoi au client le document (status=200) ou d'une erreur (status=404 quand le document n'existe pas)
 - Déconnexion

Connexion



Méthodes du protocole HTTP

- Une requête HTTP peut être envoyée en utilisant les méthodes suivantes:
 - **GET** : Pour récupérer le contenu d'un document
 - **POST** : Pour soumissionner des formulaires (Envoyer, dans la requête, des données saisies par l'utilisateur)
 - **PUT** pour envoyer un fichier du client vers le serveur
 - **DELETE** permet de demander au serveur de supprimer un document.
 - **HEAD** permet de récupérer les informations sur un document (Type, Capacité, Date de dernière modification etc...)

Le client envoie la requête : Méthode POST

Entête de la requête

Post /Nom_Script HTTP/1.0

Accept: text/html

Accept-Language : fr

User-Agent : Mozilla/4.0

*** saut de ligne ***

login=Value1& pass=Value2
& Var3=Value3

corps de la requête

Le client envoie la requête : Méthode GET

Entête de la requête

GET /Nom_Script?login=val1&pass=val2&.... HTTP/1.0
Accept: text/html
Accept-Language : fr
User-Agent : Mozilla/4.0

corps de la requête est vide

Le Serveur retourne la réponse :

Entête de la réponse

HTTP/1.0 200 OK

Date : Wed, 05Feb02 15:02:01 GMT

Server : Apache/1.3.24

Last-Modified : Wed 02Oct01 24:05:01GMT

Content-Type : Text/html

Content-length : 4205

Ligne de Status

Date du serveur

Nom du Serveur

Dernière modification

Type de contenu

Sa taille

***** saut de ligne *****

<HTML><HEAD>

....

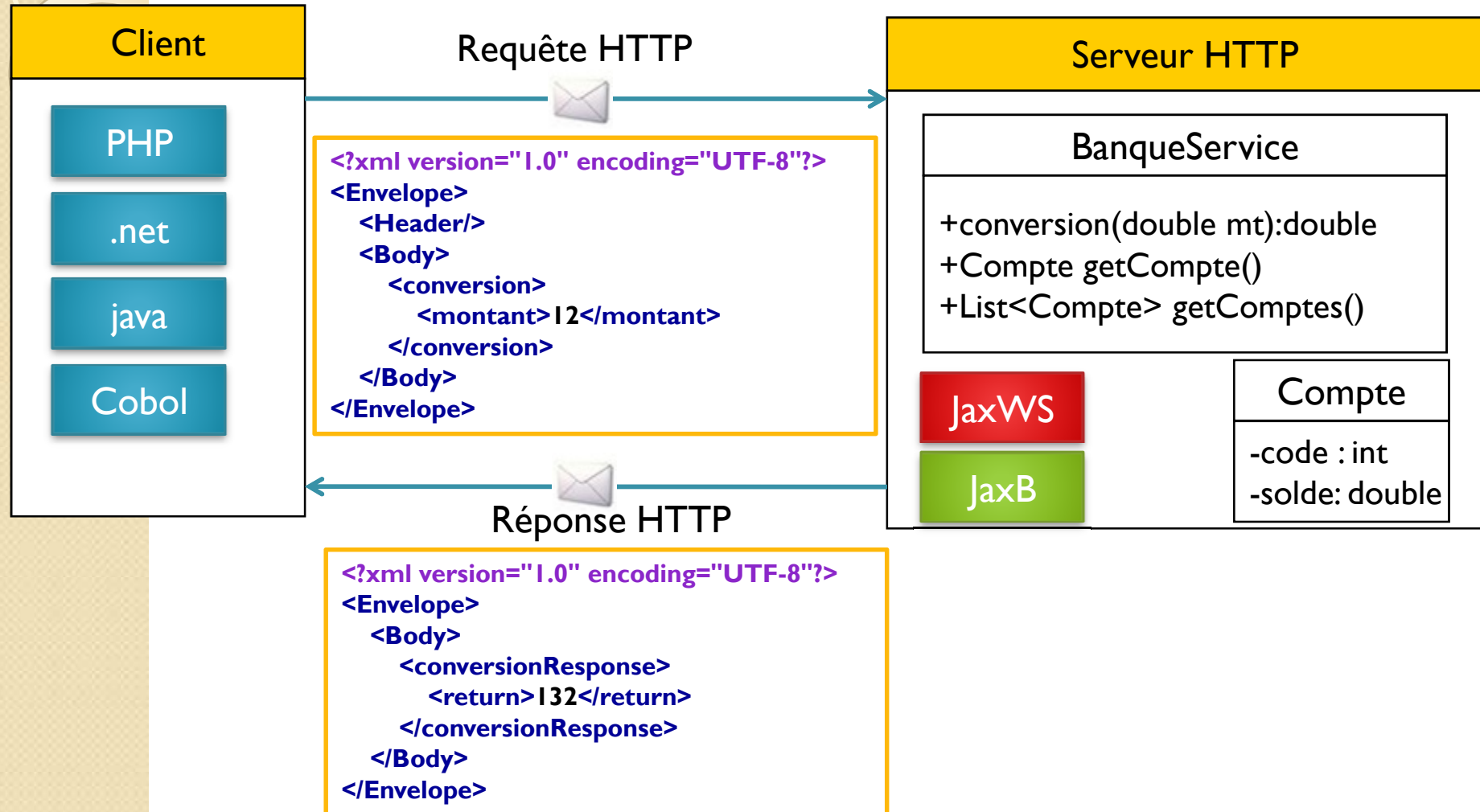
</BODY></HTML>

Le fichier que le client va afficher

corps de la réponse

net

L'idée des Web Services



Requête SOAP avec POST

Entête de la requête

Post /Nom_Script HTTP/1.0

Accept: application/xml

Accept-Language : fr

***** saut de ligne *****

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">  
  <SOAP-ENV:Header/>  
  <SOAP-ENV:Body>  
    <conversion xmlns="http://bk/test">  
      <montant xmlns="">12</montant>  
    </conversion>  
  </SOAP-ENV:Body>  
</SOAP-ENV:Envelope>
```

corps de la requête SOAP

Réponse SOAP :

Entête de la réponse

HTTP/1.0 200 OK

Date :Wed, 05Feb02 15:02:01 GMT

Server :Apache/1.3.24

Mime-Version 1.0

Last-Modified :Wed 02Oct01 24:05:01 GMT

Content-Type :Text/xml

Content-length : 4205

***** saut de ligne *****

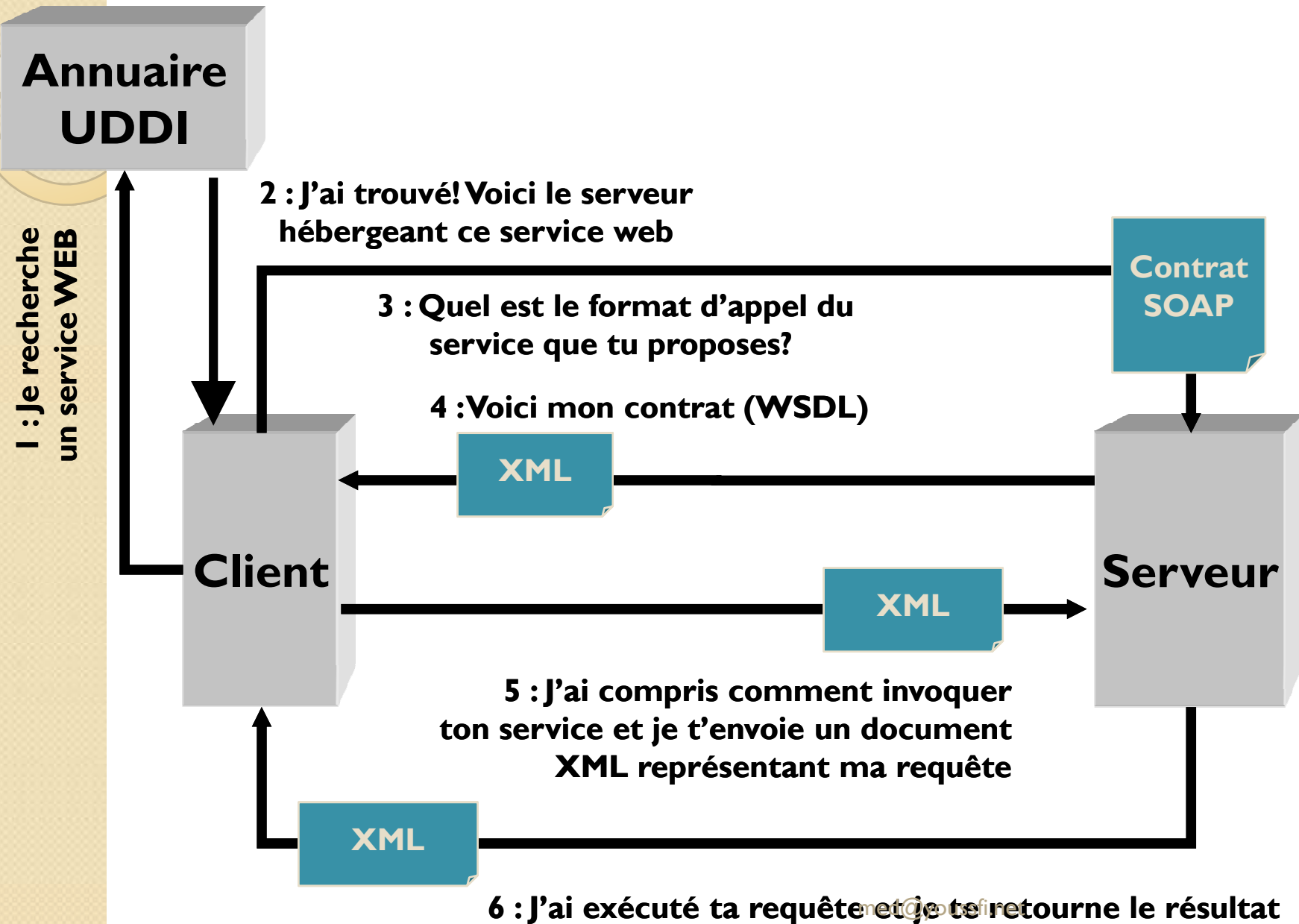
```
<?xml version="1.0" ?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:ns1="http://bk/test">
  <soapenv:Body>
    <ns1:conversionResponse>
      <return>132.0</return>
    </ns1:conversionResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

corps de la réponse

Concepts des web services

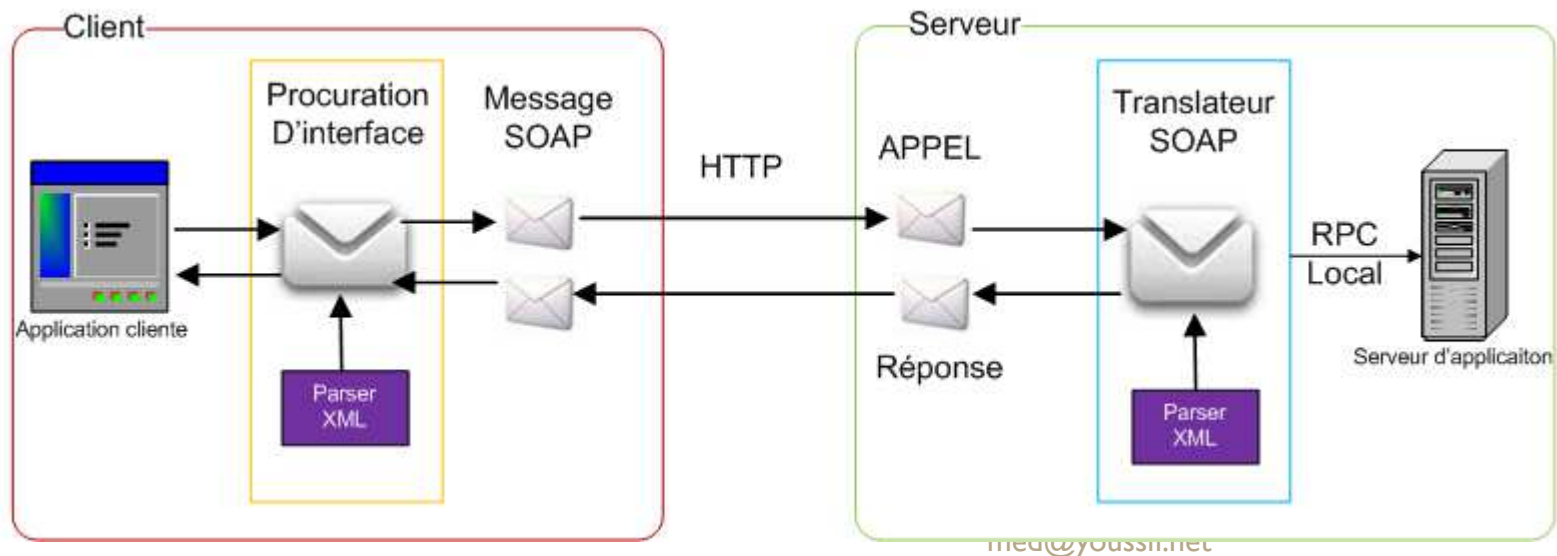
- Le concept des Web Services s'articule actuellement autour des trois concepts suivants :
 - SOAP (*Simple Object Access Protocol*)
 - est un protocole d'échange inter-applications indépendant de toute plate-forme, basé sur le langage XML.
 - Un appel de service SOAP est un flux ASCII encadré dans des balises XML et transporté dans le protocole HTTP.
 - WSDL (*Web Services Description Language*)
 - donne la description au format XML des Web Services en précisant les méthodes pouvant être invoquées, leurs signatures et le point d'accès (URL, port, etc..).
 - C'est, en quelque sorte, l'équivalent du langage IDL pour la programmation distribuée CORBA.
 - UDDI (*Universal Description, Discovery and Integration*)
 - normalise une solution d'annuaire distribué de Web Services, permettant à la fois la publication et l'exploration (recherche) de Web Services.
 - UDDI se comporte lui-même comme un Web service dont les méthodes sont appelées via le protocole SOAP.

Cycle de vie d'utilisation

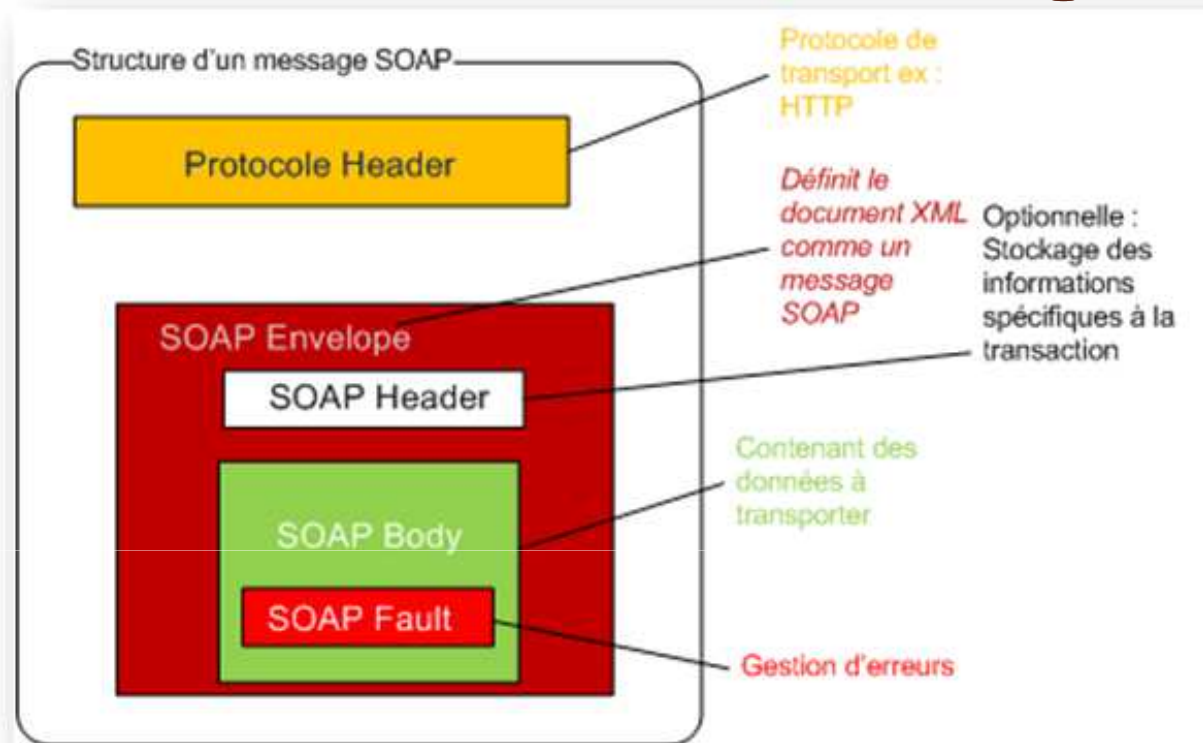


SOAP

- SOAP est un protocole d'invocation de méthodes sur des services distants. **Basé sur XML**,
- SOAP a pour principal objectif **d'assurer la communication entre machines**.
- Le protocole permet d'appeler une méthode RPC et d'envoyer des messages aux machines distantes via un protocole de transport (HTTP).
- Ce protocole est très bien adapté à l'utilisation des services Web, car il permet de fournir au client une grande quantité d'informations récupérées sur un réseau de serveurs tiers, voyez :



Structure d'un message SOAP



```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding">
  <soap:Header>
    <!-- en-tête -->
  </soap:Header>
  <soap:Body>
    <!-- corps -->
  </soap:Body>
</soap:Envelope>
```

Structure d'un message SOAP

- **SOAP envelope** (enveloppe)
 - Est l'élément de base du message SOAP.
 - L'enveloppe contient la spécification des espaces de désignation (namespace) et du codage de données.
- **SOAP header**
 - (entête) est une partie facultative qui permet d'ajouter des fonctionnalités à un message SOAP de manière décentralisée sans agrément entre les parties qui communiquent.
 - L'entête est utile surtout, quand le message doit être traité par plusieurs intermédiaires.
- **SOAP body** (corps) est un *container* pour les informations mandataires à l'intention du récepteur du message, il contient les méthodes et les paramètres qui seront exécutés par le destinataire final.
- **SOAP fault** (erreur) est un élément facultatif défini dans le corps SOAP et qui est utilisé pour reporter les erreurs.

Mise en œuvre des web services avec JAX-WS

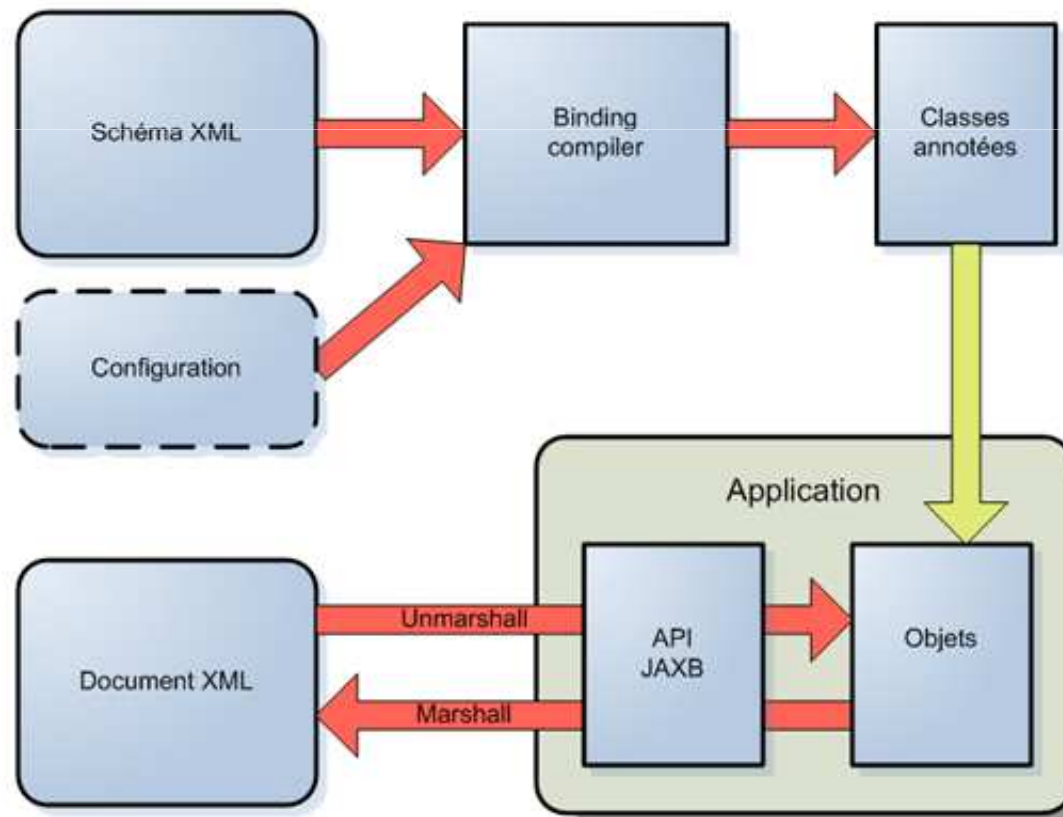
JAX-WS

- JAX-WS est la nouvelle appellation de JAX-RPC (Java API for XML Based RPC) qui permet de développer très simplement des services web en Java.
- JAX-WS fournit un ensemble d'annotations pour mapper la correspondance Java-WSDL. Il suffit pour cela d'annoter directement les classes Java qui vont représenter le service web.
- Dans l'exemple ci-dessous, une classe Java utilise des annotations JAX-WS qui vont permettre par la suite de générer le document WSDL. Le document WSDL est auto-généré par le serveur d'application au moment du déploiement :

```
@WebService(serviceName="BanqueWS")
public class BanqueService {
    @WebMethod(operationName="ConversionEuroToDh")
    public double conversion(@WebParam(name="montant")double mt){
        return mt*11;
    }
    @WebMethod
    public String test(){    return "Test";    }
    @WebMethod
    public Compte getCompte(){ return new Compte (1,7000); }
    @WebMethod
    public List<Compte> getComptes(){
        List<Compte> cptes=new ArrayList<Compte>();
        cptes.add (new Compte (1,7000)); cptes.add (new Compte (2,9000));
        return cptes;
    }
}
```

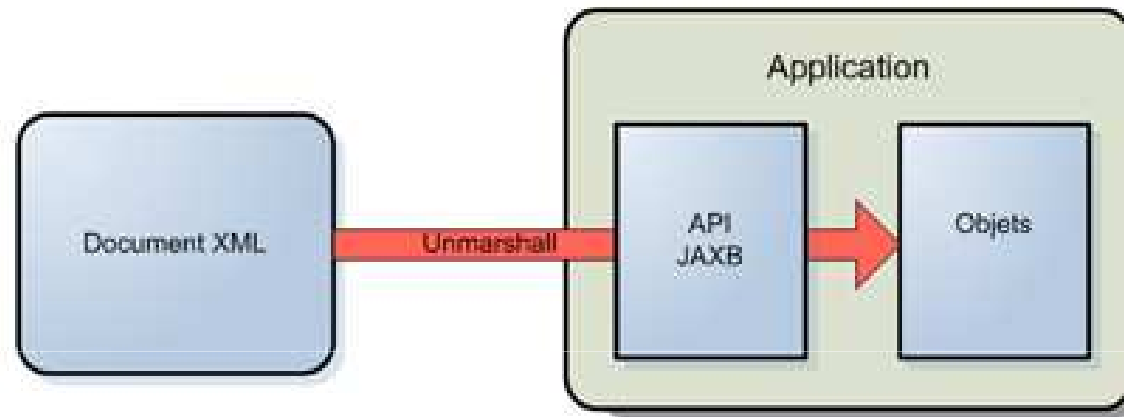

JAX-WS / JAXB

- JAX-WS s'appuie sur l'API JAXB 2.0 pour tout ce qui concerne la correspondance entre document XML et objets Java.
- JAXB 2.0 permet de mapper des objets Java dans un document XML et vice versa.
- Il permet aussi de générer des classes Java à partir un schéma XML et vice versa.

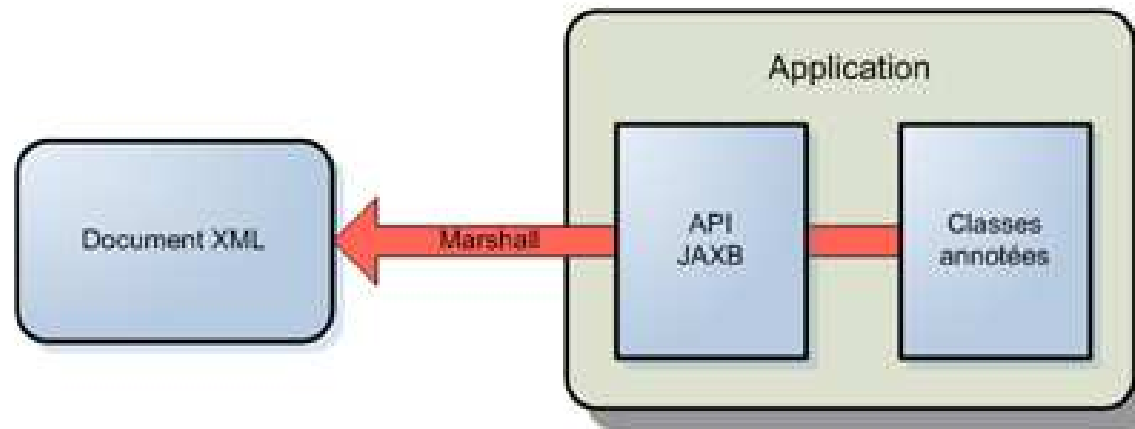


Principe de JAXB

- Le mapping d'un document XML à des objets (unmarshal)

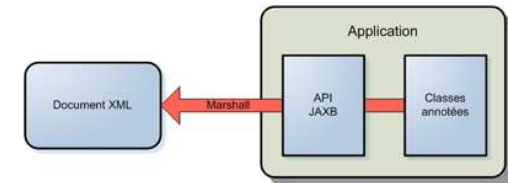


- La création d'un document XML à partir d'objets (marshal)



Générer XML à partir des objet java avec JAXB

```
package ws;
import java.io.File; import java.util.Date;
import javax.xml.bind.*;
public class Banque {
public static void main(String[] args) throws Exception {
    JAXBContext context=JAXBContext.newInstance(Compte.class);
    Marshaller marshaller=context.createMarshaller();
    marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT,true);
    Compte cp=new Compte(1,8000,new Date());
    marshaller.marshal(cp,new File("comptes.xml"));
}}
```



```
package ws;
import java.util.Date;
import javax.xml.bind.annotation.*;
@XmlRootElement
public class Compte {
    private int code;
    private float solde;
    private Date dateCreation;
    // Constructeur sans paramètre
    // Constructeur avec paramètres
    // Getters et Setters
}
```

Fichier XML Généré : comptes.xml

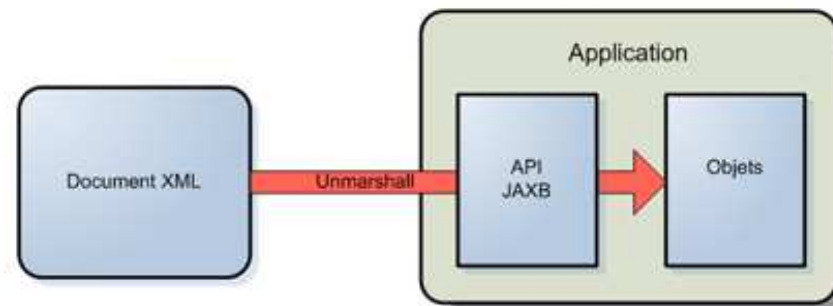
```
<?xml version="1.0" encoding="UTF-8"?>
<compte>
  <code>1</code>
  <dateCreation>
    2014-01-16T12:33:16.960Z
  </dateCreation>
  <solde>8000.0</solde>
</compte>
```

Générer des objets java à partir des données XML

```
package ws;
import java.io.*;
import javax.xml.bind.*;
public class Banque2 {
public static void main(String[] args) throws Exception {
    JAXBContext jc=JAXBContext.newInstance(Compte.class);
    Unmarshaller unmarshaller=jc.createUnmarshaller();
    Compte cp=(Compte) unmarshaller.unmarshal(new File("comptes.xml"));
    System.out.println(cp.getCode()+"-"+cp.getSolde()+"-
"+cp.getDateCreation());
}
}
```

Fichier XML Source : comptes.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<compte>
  <code>1</code>
  <dateCreation>
    2014-01-16T12:33:16.960Z
  </dateCreation>
  <solde>8000.0</solde>
</compte>
```



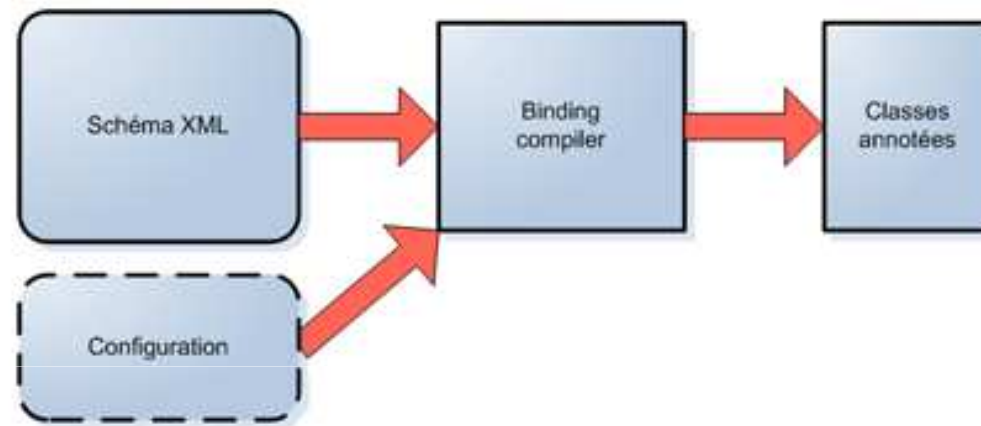
Générer un schéma XML à partir d'une classe avec JAXB

```
package ws;
import java.io.*;
import javax.xml.bind.*;import javax.xml.transform.Result;
import javax.xml.transform.stream.StreamResult;
public class Banque {
    public static void main(String[] args) throws Exception {
        JAXBContext context=JAXBContext.newInstance(Compte.class);
        context.generateSchema(new SchemaOutputResolver() {
            @Override
            public Result createOutput(String namespaceUri, String suggestedFileName)
            throws IOException {
                File f=new File("compte.xsd");
                StreamResult result=new StreamResult(f);
                result.setSystemId(f.getName());
                return result;
            }
        });
    }
}
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<xs:schema version="1.0" xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="compte" type="compte"/>
  <xs:complexType name="compte">
    <xs:sequence>
      <xs:element name="code" type="xs:int"/>
      <xs:element name="dateCreation" type="xs:dateTime" minOccurs="0"/>
      <xs:element name="solde" type="xs:float"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

Génération des classes à partir d'un schéma XML

- Pour permettre l'utilisation et la manipulation d'un document XML, JAXB propose de générer un ensemble de classes à partir du schéma XML du document.



- L'implémentation de référence fournit l'outil **xjc** pour générer les classes à partir d'un schéma XML.
- L'utilisation la plus simple de l'outil **xjc** est de lui fournir simplement le fichier qui contient le schéma XML du document à utiliser.

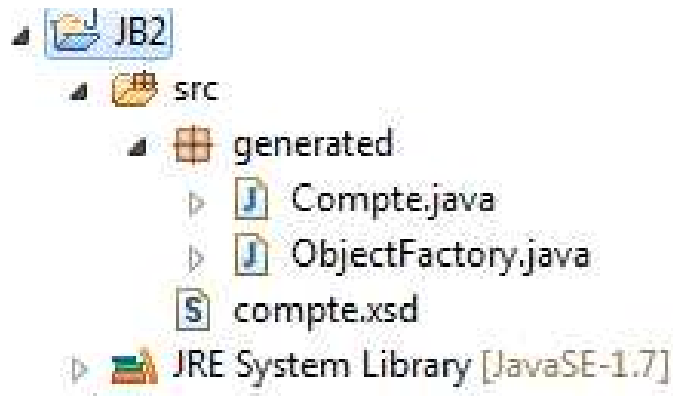
```
C:\Windows\system32\cmd.exe

C:\Users\youssfi\w\JB2\src>xjc compte.xsd
parsing a schema...
compiling a schema...
generated\Compte.java
generated\ObjectFactory.java

C:\Users\youssfi\w\JB2\src>
```

Classes générées par XJC

- L'outil XJC génère deux classes :
 - Compte.java qui correspond au type complexe Compte dans le schéma xml.
 - ObjectFactory.java : une fabrique qui permet de créer des objets de type Compte.



Quelques annotations JAXB

Annotation	Description
@XmlRootElement	Associer une classe ou une énumération à un élément XML
@XmlSchema	Associer un espace de nommage à un package
@XmlTransient	Marquer une entité pour ne pas être mappée dans le document XML
@XmlAttribute	Convertir une propriété en un attribut dans le document XML
@XmlElement	Convertir une propriété en un élément dans le document XML
@XmlAccessorType	Préciser comment un champ ou une propriété est sérialisé
@XmlNs	Associer un préfixe d'un espace de nommage à un URI

Comment développer un web service JAW-WS

1. Créer le service Web

- Développer le web service
- Déployer le web service
 - Un serveur HTTP
 - Un Conteneur WS (JaxWS, AXIS, CXF, etc...)

2. Tester le web service avec un analyseur SOAP

- SoapUI,
- Oxygen, etc...

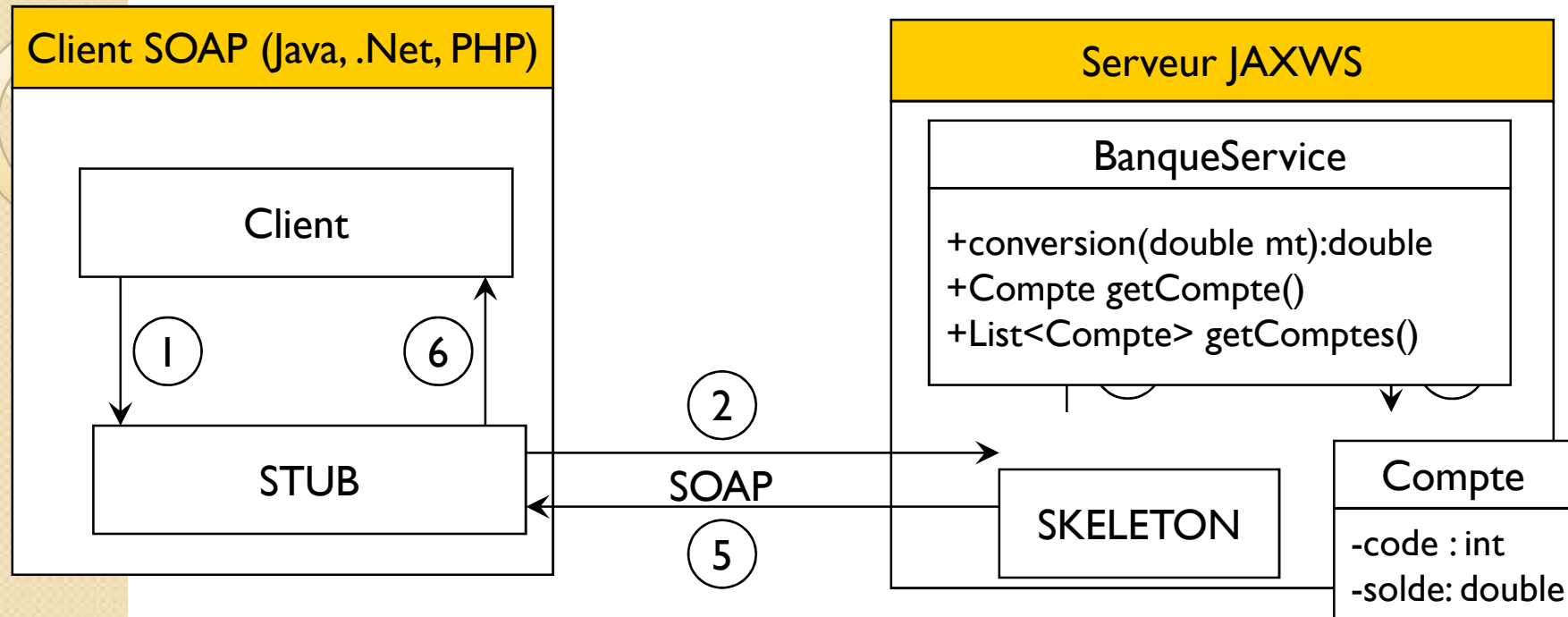
3. Créer les clients :

- Un client Java
- Un client .Net
- Un client PHP

Application

- Créer un web service java en utilisant JaxWS qui permet de :
 - Convertir un montant de l'auro en DH
 - Consulter un compte sachant son code
 - Consulter une liste de comptes
- Tester le web service avec un analyseur SOAP
- Créer un client Java
- Créer un client .Net
- Créer un client PHP

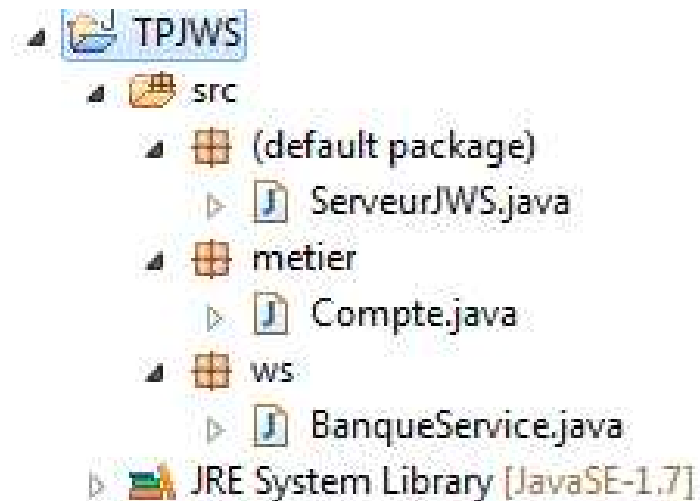
Architecture I



- 1 Le client demande au stub de faire appel à la méthode conversion(12)
- 2 Le Stub se connecte au Skeleton et lui envoie une requête SOAP
- 3 Le Skeleton fait appel à la méthode du web service
- 4 Le web service retourne le résultat au Skeleton
- 5 Le Skeleton envoie le résultat dans une la réponse SOAP au Stub
- 6 Le Stub fournie le résultat au client

Développer le web service avec JAX-WS

- Outils à installer :
 - JDK 1.7
 - Editeur java : Eclipse.
- Créer un projet java en utilisant JDK 1.7 comme compilateur et environnement d'exécution java.
 - Structure du projet :



Implémentation de la classe Compte

```
package metier;
import java.util.Date;
import javax.xml.bind.annotation.XmlRootElement;
import javax.xml.bind.annotation.XmlTransient;
@XmlRootElement
@XmlAccessorType(XmlAccessType.FIELD)
public class Compte {
    private Long code;
    private double solde;
    @XmlTransient
    private Date dateCreation;
    // Constructeur sans paramètre et avec paramètre
    // Getters et setters
}
```

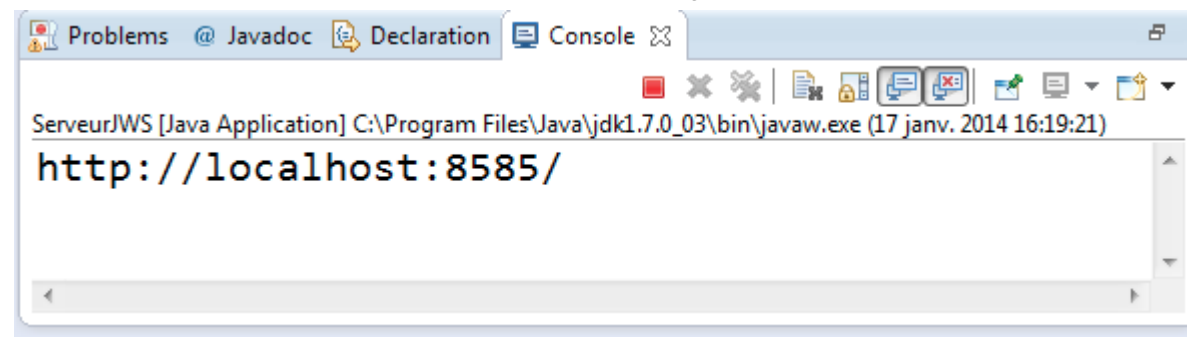
Implémentation du web service

```
package ws;
import java.util.*;
import javax.jws.*;
import metier.Compte;
@WebService(serviceName="BanqueWS")
public class BanqueService {
    @WebMethod(operationName="ConversionEuroToDh")
    public double conversion(@WebParam(name="montant")double mt){
        return mt*11;
    }
    @WebMethod
    public Compte getCompte(@WebParam(name="code")Long code){
        return new Compte (code,7000,new Date());
    }
    @WebMethod
    public List<Compte> getComptes(){
        List<Compte> cptes=new ArrayList<Compte>();
        cptes.add (new Compte (1L,7000,new Date()));
        cptes.add (new Compte (2L,7000,new Date()));
        return cptes;
    }
}
```

Simple Serveur JAX WS

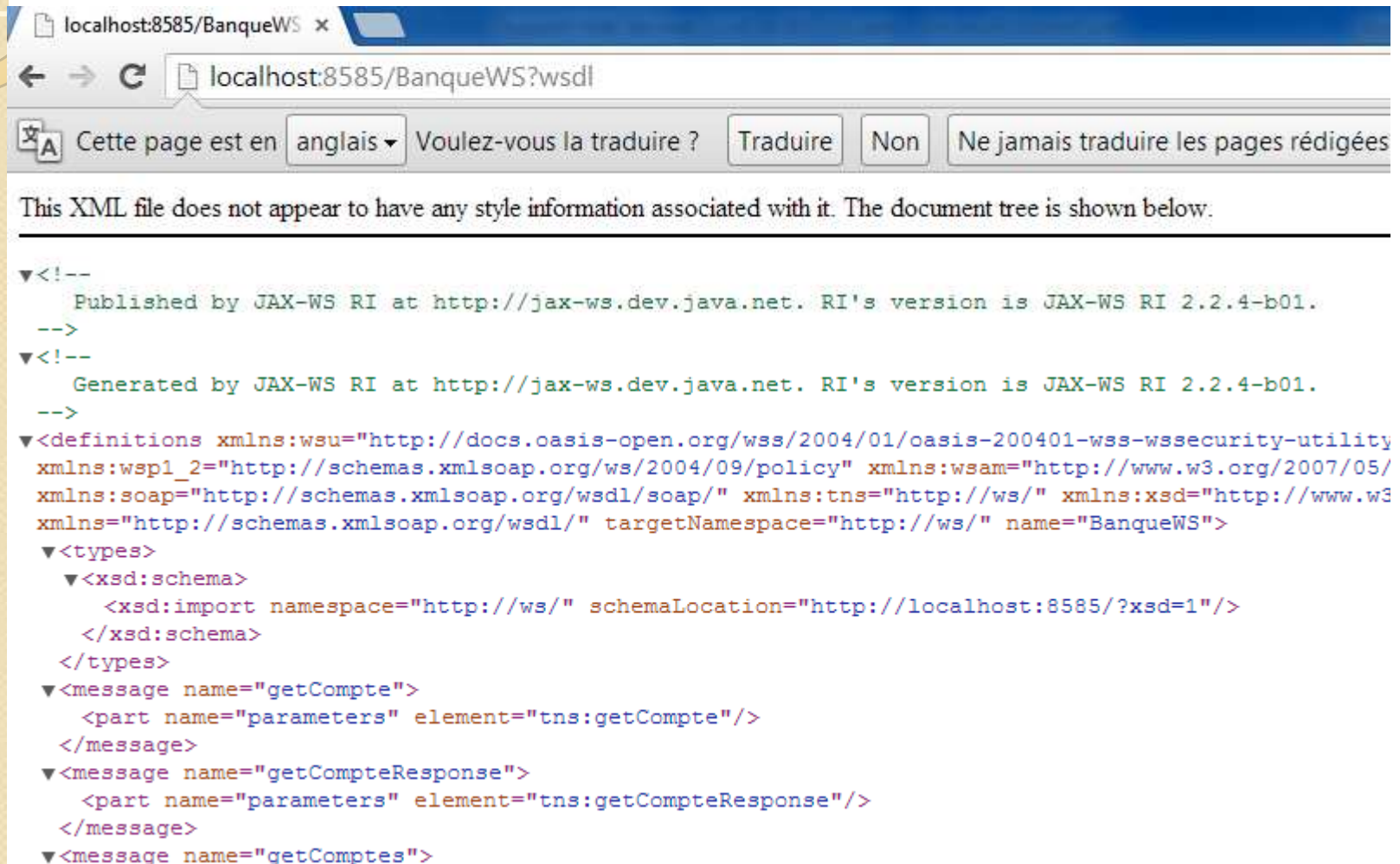
```
import javax.xml.ws.Endpoint;  
import ws.BanqueService;  
public class ServeurJWS {  
    public static void main(String[] args) {  
        String url="http://localhost:8585/";  
        Endpoint.publish(url, new BanqueService());  
        System.out.println(url);  
    }  
}
```

Exécution du serveur Java



Analyser le WSDL

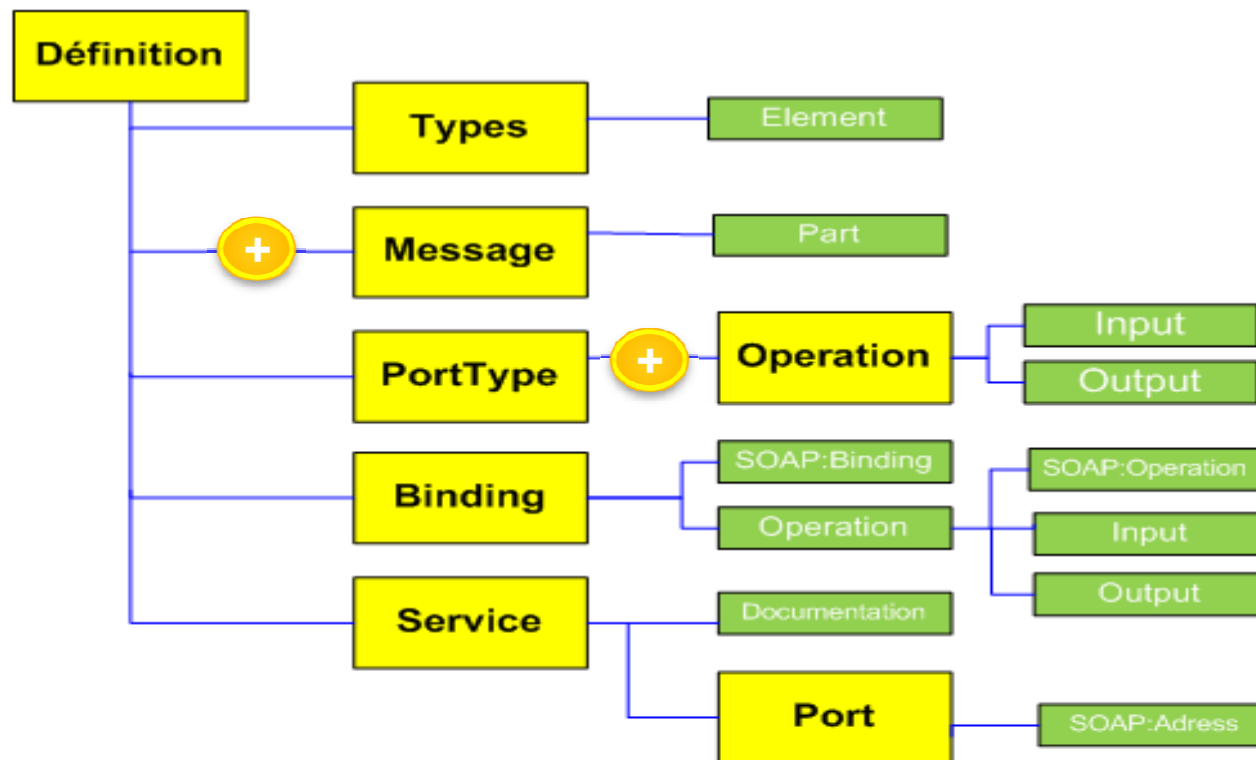
- Pour Visualiser le WSDL, vous pouvez utiliser un navigateur web



```
▼<!--
  Published by JAX-WS RI at http://jax-ws.dev.java.net. RI's version is JAX-WS RI 2.2.4-b01.
-->
▼<!--
  Generated by JAX-WS RI at http://jax-ws.dev.java.net. RI's version is JAX-WS RI 2.2.4-b01.
-->
▼<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility
xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http://www.w3.org/2007/05/
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:tns="http://ws/" xmlns:xsd="http://www.w3
xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://ws/" name="BanqueWS">
  ▼<types>
    ▼<xsd:schema>
      <xsd:import namespace="http://ws/" schemaLocation="http://localhost:8585/?xsd=1"/>
    </xsd:schema>
  </types>
  ▼<message name="getCompte">
    <part name="parameters" element="tns:getCompte"/>
  </message>
  ▼<message name="getCompteResponse">
    <part name="parameters" element="tns:getCompteResponse"/>
  </message>
  ▼<message name="getComptes">
```

Structure du WSDL

- Un document WSDL se compose d'un ensemble d'éléments décrivant les types de données utilisés par le service, les messages que le service peut recevoir, ainsi que les liaisons SOAP associées à chaque message.
- Le schéma suivant illustre la structure du langage WSDL qui est un document XML, en décrivant les relations entre les sections constituant un document WSDL.



Structure d'un document WSDL

Structure du WSDL

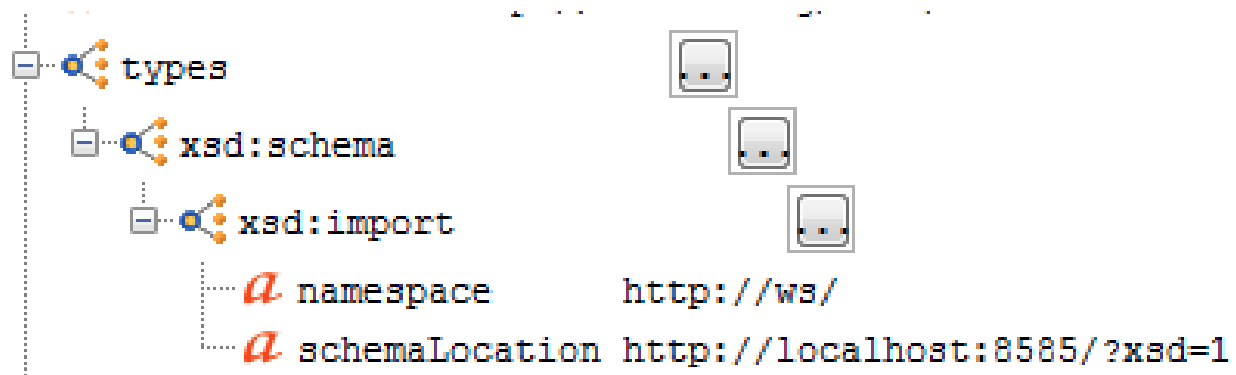
- Un fichier WSDL contient donc sept éléments.
 - **Types** : fournit la définition de types de données utilisés pour décrire les messages échangés.
 - **Messages** : représente une définition abstraite (noms et types) des données en cours de transmission.
 - **PortTypes** : décrit un ensemble d'opérations. Chaque opération a zéro ou un message en entrée, zéro ou plusieurs messages de sortie ou d'erreurs.
 - **Binding** : spécifie une liaison entre un <portType> et un protocole concret (SOAP, HTTP...).
 - **Service** : indique les adresses de port de chaque liaison.
 - **Port** : représente un point d'accès de services défini par une adresse réseau et une liaison.
 - **Opération** : c'est la description d'une action exposée dans le port.

Structure du WSDL

Document - Banque.wSDL

definitions	
name	BanqueWS
targetNamespace	http://ws/
xmlns	http://schemas.xmlsoap.org/wsdl/
xmlns:soap	http://schemas.xmlsoap.org/wsdl/soap/
xmlns:tns	http://ws/
xmlns:wsam	http://www.w3.org/2007/05/addressing/metadata
xmlns:wsp	http://www.w3.org/ns/ws-policy
xmlns:wsp1_2	http://schemas.xmlsoap.org/ws/2004/09/policy
xmlns:wsu	http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd
xmlns:xsd	http://www.w3.org/2001/XMLSchema
types	
message	
message	
message	
message	
message	
message	
message	
portType	
binding	
service	

Elément Types



XML Schema

← → ↻ localhost:8585/?xsd=1

📄 Cette page est en anglais ▼ Voulez-vous la traduire ? Traduire Non Ne jamais traduire les pages rédigées en anglais

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<!--
  Published by JAX-WS RI at http://jax-ws.dev.java.net. RI's version is JAX-WS RI 2.2.4-b01.
-->
<xs:schema xmlns:tns="http://ws/" xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0" targetNamespace="http://ws/">
  <xs:element name="ConversionEuroToDh" type="tns:ConversionEuroToDh"/>
  <xs:element name="ConversionEuroToDhResponse" type="tns:ConversionEuroToDhResponse"/>
  <xs:element name="compte" type="tns:compte"/>
  <xs:element name="getCompte" type="tns:getCompte"/>
  <xs:element name="getCompteResponse" type="tns:getCompteResponse"/>
  <xs:element name="getComptes" type="tns:getComptes"/>
  <xs:element name="getComptesResponse" type="tns:getComptesResponse"/>
  <xs:complexType name="ConversionEuroToDh">
    <xs:sequence>
      <xs:element name="montant" type="xs:double"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="ConversionEuroToDhResponse">
    <xs:sequence>
      <xs:element name="return" type="xs:double"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="getCompte">
    <xs:sequence>
      <xs:element name="code" type="xs:long" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="getCompteResponse">
    <xs:sequence>
      <xs:element name="return" type="tns:compte" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="compte">
    <xs:sequence>
      <xs:element name="code" type="xs:long" minOccurs="0"/>
      <xs:element name="solde" type="xs:double"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="getComptes">
    <xs:sequence/>
  </xs:complexType>
  <xs:complexType name="getComptesResponse">
    <xs:sequence>
      <xs:element name="return" type="tns:compte" minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

XML Schema

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:tns="http://ws/" xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0"
  targetNamespace="http://ws/">
  <xs:element name="ConversionEuroToDh" type="tns:ConversionEuroToDh"></xs:element>
  <xs:element name="ConversionEuroToDhResponse" type="tns:ConversionEuroToDhResponse"/>
  <xs:element name="compte" type="tns:compte"></xs:element>
  <xs:element name="getCompte" type="tns:getCompte"></xs:element>
  <xs:element name="getCompteResponse" type="tns:getCompteResponse"></xs:element>
  <xs:element name="getComptes" type="tns:getComptes"></xs:element>
  <xs:element name="getComptesResponse" type="tns:getComptesResponse"></xs:element>

  <xs:complexType name="ConversionEuroToDh">
    <xs:sequence>
      <xs:element name="montant" type="xs:double"></xs:element>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="ConversionEuroToDhResponse">
    <xs:sequence>
      <xs:element name="return" type="xs:double"></xs:element>
    </xs:sequence>
  </xs:complexType>
```

XML Schema

```
<xs:complexType name="getCompte">
  <xs:sequence>
    <xs:element name="code" type="xs:long" minOccurs="0"></xs:element>
  </xs:sequence>
</xs:complexType>

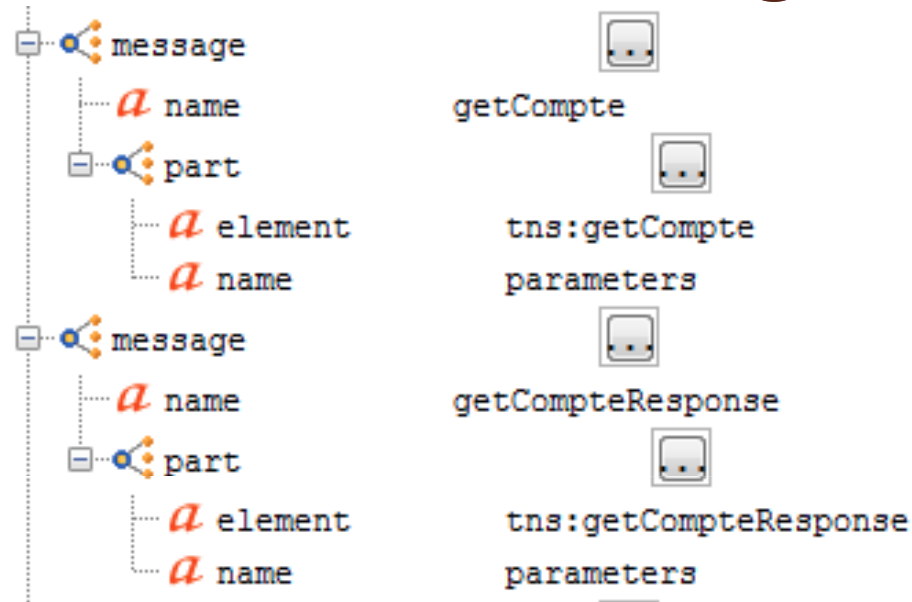
<xs:complexType name="getCompteResponse">
  <xs:sequence>
    <xs:element name="return" type="tns:compte" minOccurs="0"></xs:element>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="compte">
  <xs:sequence>
    <xs:element name="code" type="xs:long" minOccurs="0"></xs:element>
    <xs:element name="solde" type="xs:double"></xs:element>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="getComptes">
  <xs:sequence></xs:sequence>
</xs:complexType>

<xs:complexType name="getComptesResponse">
  <xs:sequence>
    <xs:element name="return" type="tns:compte" minOccurs="0" maxOccurs="unbounded"></xs:element>
  </xs:sequence>
</xs:complexType>
</xs:schema>
```

Élément message



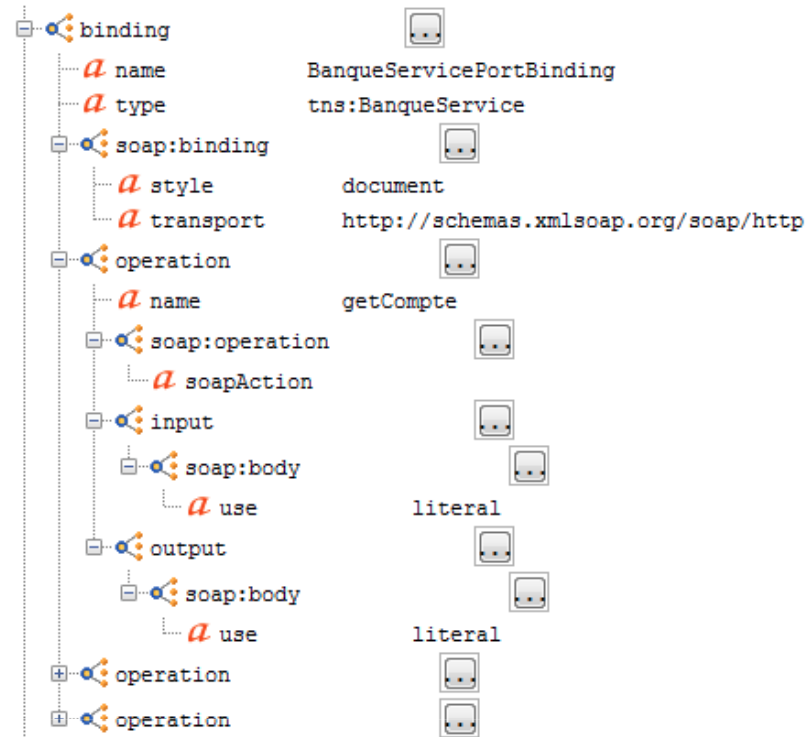
```
<message name="getCompte">  
  <part name="parameters" element="tns:getCompte"></part>  
</message>  
<message name="getCompteResponse">  
  <part name="parameters" element="tns:getCompteResponse"></part>  
</message>
```

Élément portType



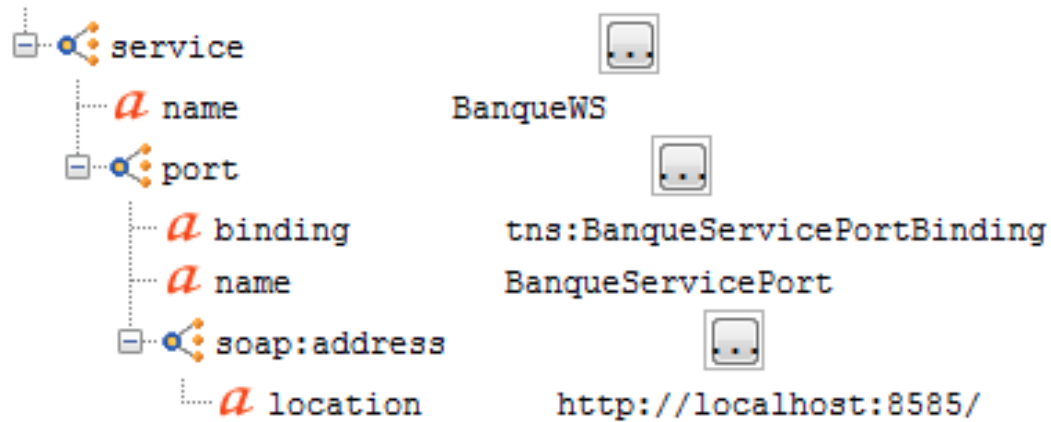
```
<portType name="BanqueService">
  <operation name="getCompte">
    <input wsam:Action="http://ws/BanqueService/getCompteRequest" message="tns:getCompte"></input>
    <output wsam:Action="http://ws/BanqueService/getCompteResponse" message="tns:getCompteResponse"></output>
  </operation>
  <operation name="getComptes">
    <input wsam:Action="http://ws/BanqueService/getComptesRequest" message="tns:getComptes"></input>
    <output wsam:Action="http://ws/BanqueService/getComptesResponse" message="tns:getComptesResponse"></output>
  </operation>
  <operation name="ConversionEuroToDh">
    ....
  </operation>
</portType>
```


Elément binding



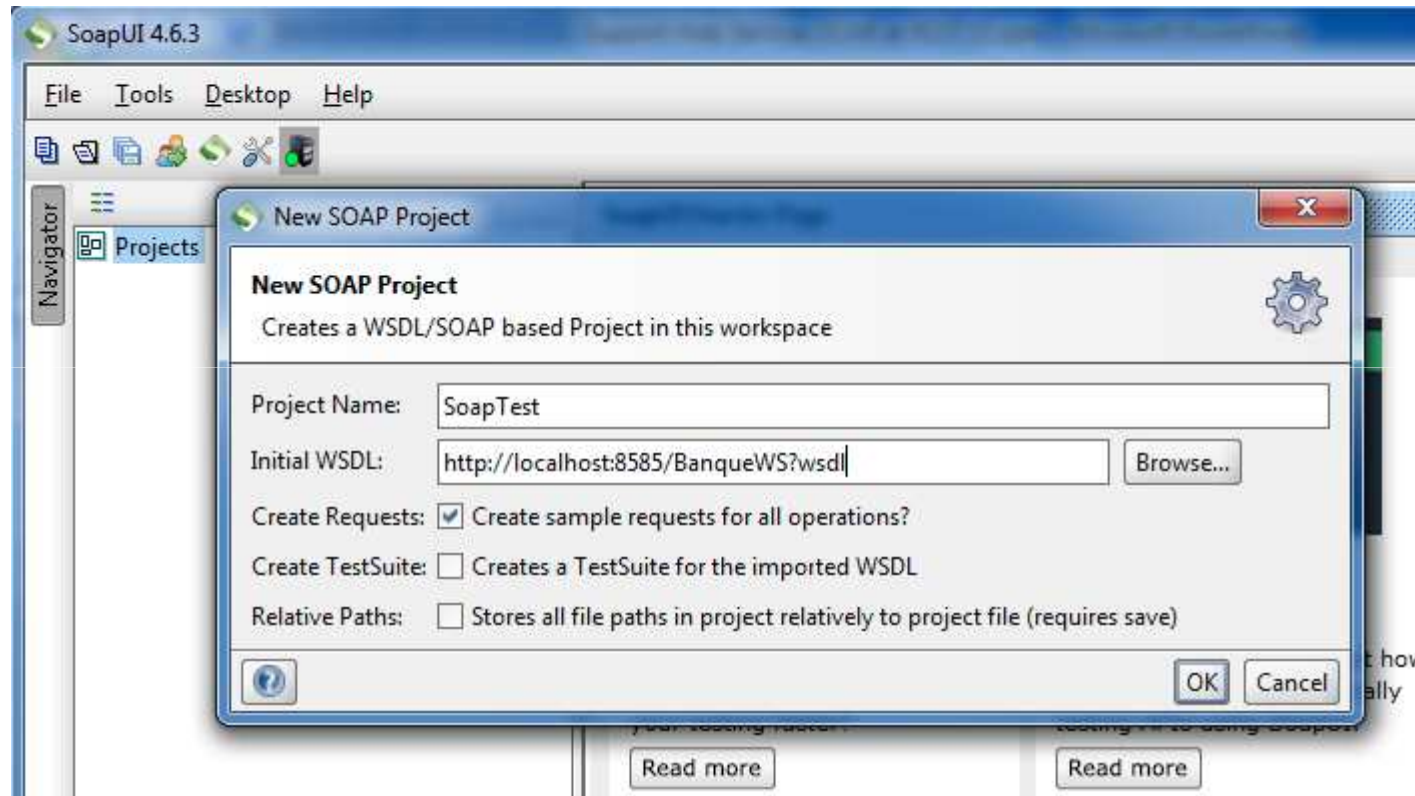
```
<binding name="BanqueServicePortBinding" type="tns:BanqueService">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"></soap:binding>
  <operation name="getCompte">
    <soap:operation soapAction=""></soap:operation>
    <input>
      <soap:body use="literal"></soap:body>
    </input>
    <output>
      <soap:body use="literal"></soap:body>
    </output>
  </operation>
  <operation name="getComptes">
    ....
  </operation>
</binding>
```

Élément service

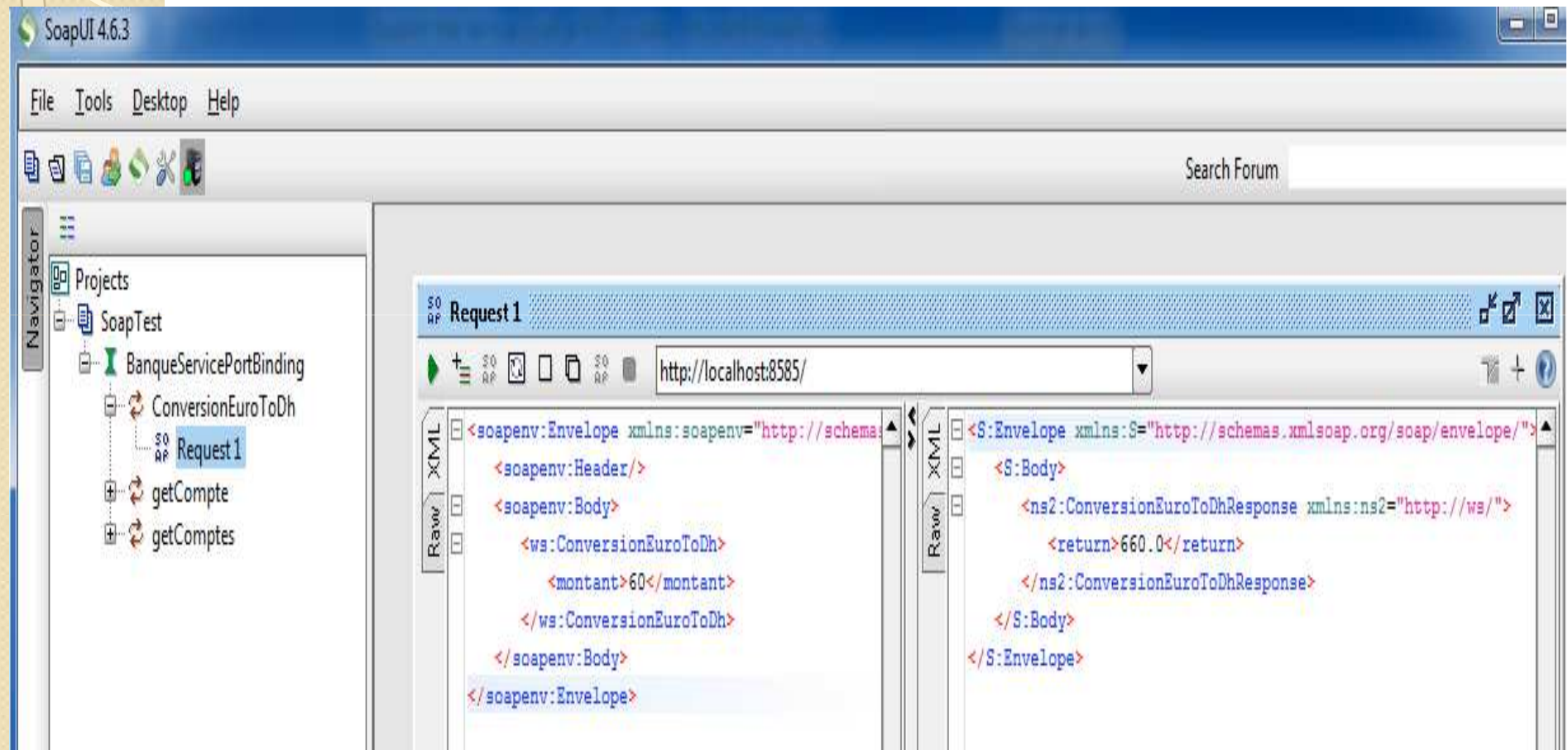


```
<service name="BanqueVVS">  
  <port name="BanqueServicePort" binding="tns:BanqueServicePortBinding">  
    <soap:address location="http://localhost:8585/"></soap:address>  
  </port>  
</service>
```

Tester les méthodes du web service avec un analyseur SOAP : SoapUI



Tester les méthodes du web service avec un analyseur SOAP : SoapUI



Tester la méthode conversion

Requête SOAP

- ```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ws="http://ws/"
 <soapenv:Header/>
 <soapenv:Body>
 <ws:ConversionEuroToDh>
 <montant>60</montant>
 </ws:ConversionEuroToDh>
 </soapenv:Body>
</soapenv:Envelope>
```

## Réponse SOAP

- ```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:ConversionEuroToDhResponse xmlns:ns2="http://ws/">
      <return>660.0</return>
    </ns2:ConversionEuroToDhResponse>
  </S:Body>
</S:Envelope>
```

Tester la méthode getCompte

Requête SOAP

- ```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ws="http://ws/">
 <soapenv:Header/>
 <soapenv:Body>
 <ws:getCompte>
 <code>2</code>
 </ws:getCompte>
 </soapenv:Body>
</soapenv:Envelope>
```

## Réponse SOAP

- ```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:getCompteResponse xmlns:ns2="http://ws/">
      <return>
        <code>2</code>
        <solde>7000.0</solde>
      </return>
    </ns2:getCompteResponse>
  </S:Body>
</S:Envelope>
```

Tester la méthode getComptes

Requête SOAP

- ```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ws="http://ws/">
 <soapenv:Header/>
 <soapenv:Body>
 <ws:getComptes/>
 </soapenv:Body>
</soapenv:Envelope>
```

## Réponse SOAP

- ```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:getComptesResponse xmlns:ns2="http://ws/">
      <return>
        <code>1</code>
        <solde>7000.0</solde>
      </return>
      <return>
        <code>2</code>
        <solde>7000.0</solde>
      </return>
    </ns2:getComptesResponse>
  </S:Body>
</S:Envelope>
```